

SQL Questions & Solutions

SQL Question 1 — Customer with the Highest Value Order

Marks: 10

Difficulty: Easy

Database: RETAIL_STORE_DB

Management wants to know which customer placed the single highest-value order. Find and return the customer's name along with the order_id and total_amount of that order.

Requirements:

- Use a single-row subquery to find the maximum total_amount in the Orders table.
- Join Orders with Customers to retrieve the customer name.
- Return the order with the maximum total_amount.

Required Output Columns: order_id, customer_name, total_amount

SQL Query:

```
SELECT o.order_id, c.name AS customer_name, o.total_amount
FROM Orders o
JOIN Customers c ON o.customer_id = c.customer_id
WHERE o.total_amount = (SELECT MAX(total_amount) FROM Orders );
```

SQL Question 2 — Student Enrollments with Grades and Department Names

Marks: 10

Difficulty: Easy

Database: UNIVERSITY_CORE_DB

Generate a report listing every student along with the courses they are enrolled in, the grades they have received, and the departments offering those courses.

Requirements:

- Join Students with Enrollments using student_id.
- Join Enrollments with Courses using course_id.
- Join Courses with Departments using department_id.
- Return one row for each student-course enrollment.

Required Output Columns: student_id, student_name, course_id, course_name, grade, department_name

SQL Query:

```
SELECT s.student_id, s.name AS student_name, c.course_id, c.course_name, e.grade, d.department_name
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
JOIN Courses c ON e.course_id = c.course_id
JOIN Departments d ON c.department_id = d.department_id;
```

SQL Question 3 — Calculate Average Assignment Score per Course

Marks: 10

Difficulty: Easy

Database: UNIVERSITY_COURSE_DB

Calculate the average assignment score per course. Join the tables to get the course name and average score.

Requirements:

- Join Courses with Assignments using course_id.
- Join Assignments with Submissions using assignment_id.
- Use AVG() to calculate the average score for each course.
- Group the result by course name.
- Sort the result by average score in descending order.

Required Output Columns: course_name, avg_score

SQL Query:

```
SELECT c.course_name, AVG(s.score) AS avg_score
FROM Courses c
JOIN Assignments a ON c.course_id = a.course_id
JOIN Submissions s ON a.assignment_id = s.assignment_id
GROUP BY c.course_name
ORDER BY avg_score DESC;
```

SQL Question 4 — Highest-Revenue Doctor per Department

Marks: 20

Difficulty: Medium

Database: HOSPITAL_DB

A hospital administration system tracks appointments and billing for doctors across multiple departments. Each doctor belongs to a specific department, and each completed appointment generates a bill recorded in the Bills table. Generate a report showing the top-earning doctors within each department, based on total billed amount.

Requirements:

- Calculate the total revenue generated by every doctor.
- Use RANK(), not ROW_NUMBER(), so doctors with equal total revenue receive the same rank.
- Highest revenue must have rank 1.
- Rank doctors separately within each department.

Required Output Columns: department_name, doctor_name, total_revenue, revenue_rank

SQL Query:

```
SELECT d.department_name, doc.doctor_name, SUM(b.amount) AS total_revenue,
       RANK() OVER (PARTITION BY d.dept_id ORDER BY SUM(b.amount) DESC) AS revenue_rank
FROM Departments d
JOIN Doctors doc ON d.dept_id = doc.dept_id
JOIN Appointments a ON doc.doctor_id = a.doctor_id
JOIN Bills b ON a.appointment_id = b.appointment_id
GROUP BY d.department_name, d.dept_id, doc.doctor_name
ORDER BY d.dept_id, revenue_rank;
```

SQL Question 5 — Students Ranked by Course Enrollments

Marks: 20

Difficulty: Medium

Database: UNIVERSITY_CORE_DB

Write a query to assign a RANK and a DENSE_RANK to each student based on the number of courses they are enrolled in, in descending order. If multiple students are enrolled in the same number of courses, they should receive the same rank.

Requirements:

- Count the total number of course enrollments for each student.
- Use RANK() ordered by course_count descending.
- Use DENSE_RANK() ordered by course_count descending.
- Return both ranking values.
- Order the final result by rank_by_enrollments.

Required Output Columns: student_id, name, course_count, rank_by_enrollments, dense_rank_by_enrollments

SQL Query:

```
SELECT s.student_id,s.name, COUNT(e.course_id) AS course_count,
       RANK() OVER (ORDER BY COUNT(e.course_id) DESC) AS rank_by_enrollments,
       DENSE_RANK() OVER (ORDER BY COUNT(e.course_id) DESC) AS dense_rank_by_enrollments
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
GROUP BY s.student_id, s.name
ORDER BY rank_by_enrollments;
```

SQL Question 6 — Movies Watched by Viewers of 'Life of Pi'

Marks: 20

Difficulty: Medium

Database: MOVIE_STREAMING_DB

List all movies that have been watched by any user who has watched 'Life of Pi'. Duplicate movie titles must not appear in the result.

Requirements:

- Find the users who have watched 'Life of Pi'.
- Return movies watched by those users.
- Use DISTINCT so duplicate movie titles do not appear.
- Return movie_id and title.

Required Output Columns: movie_id, title

SQL Query:

```
SELECT DISTINCT m.movie_id,m.title
FROM Movies m
JOIN Watch_History wh ON m.movie_id = wh.movie_id
WHERE wh.user_id IN (
    SELECT wh2.user_id
    FROM Watch_History wh2
    JOIN Movies m2 ON wh2.movie_id = m2.movie_id
    WHERE m2.title = 'Life of Pi'
);
```